AeroFlow: Accelerating Serverless Workflows through Serialization-Free WebAssembly Execution

Wang Xiong, Zhida Jiang, Zhuocheng Ge, Xi Wang, Chuan Luo,
Tianyu Wo, Chunming Hu, Renyu Yang[1†]
Beihang University
Beijing, China

Abstract

Serverless workflow systems orchestrate directed acyclic graphs (DAGs) of short-lived functions to build complex data-processing applications. Conventional designs rely on external storage services to transfer intermediate state between dependent functions, incurring substantial serialization and deserialization overhead on each data-dependency edge. These costs accumulate along the workflow's critical path and grow super-linearly under concurrency as parallel functions competing for shared storage induce severe resource contention. We present AeroFlow, a low-latency serverless workflow engine that eliminates serialization overhead from inter-function state transfer. AeroFlow uses lightweight, fast-startup WebAssembly sandboxes that intercept and redirect function state accesses at the host-function boundary without requiring changes to user application code. For co-located functions, this replaces storage-mediated communication with serialization-free shared-memory data exchange, yielding notable per-edge transfer latency reduction. A location-aware mixed-integer programming scheduler further optimizes function placement and data routing, co-locating data-intensive function pairs to maximize the fraction of edges served via the fast local path. On both scientific workflow benchmarks and real-world serverless workloads, AeroFlow achieves an end-to-end performance improvement of 2.4–12.4× over state-of-the-art systems at moderate scales. At a scale of 250 functions, the speedup reaches 7.8×, as baseline systems exhibit super-linear performance degradation; at a scale of 500 functions, AeroFlow is the only system that successfully completes execution.

CCS Concepts

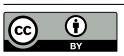
• **Computer systems organization** → **Cloud computing**; *Distributed architectures.*

Keywords

Serverless Computing, Function-as-a-Service, WebAssembly, Function State Transfer, Workflow Scheduling, Mixed-Integer Programming

ACM Reference Format:
Wang Xiong, Zhida Jiang, Zhuocheng Ge, Xi Wang, Chuan Luo, Tianyu Wo, Chunming Hu, Renyu Yang. 2026. AeroFlow: Accelerating Serverless Workflows through Serialization-Free WebAssembly Execution. In *Proceedings of the 55th International Conference on Parallel Processing (ICPP '26), September 28-October 01, 2026, Singapore, Singapore.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3832810.3832905

Corresponding Author: Renyu Yang (renyuyang@buaa.edu.cn).



ICPP '26, Singapore, Singapore

ACM ISBN 979-8-4007-2657-6/2026/09
https://doi.org/10.1145/3832810.3832905

1 Introduction

Serverless computing has emerged as a widely adopted execution paradigm for constructing complex applications from fine-grained, stateless functions orchestrated as workflows [1, 2]. A serverless workflow can be formally modeled as a directed acyclic graph (DAG), in which each vertex represents a short-lived function and each edge represents a data dependency. This dependency implies that a downstream function cannot be invoked until it has received the output produced by its upstream function. Owing to the stateless nature of these functions by design, conventional workflow engines persist all intermediate results in external storage systems, such as Redis, CouchDB, or Kafka brokers. Along each dependency edge, this *storage-mediated* communication pattern induces a three-stage pipeline (Figure 1): the producer first *serializes* its output into a wire format, then *transmits* the serialized payload to the storage service, and finally the consumer *deserializes* the payload before continuing execution. Our controlled decomposition (§2) demonstrates that serialization accounts for 80–90% of the per-edge Redis transfer latency for payloads ranging from 1 to 50 MB, substantially exceeding the latency attributable to raw storage I/O. These per-edge overheads accumulate along the DAG critical path and are further exacerbated under concurrent execution: as many parallel functions simultaneously write to the shared storage tier, lock contention and throughput saturation cause the per-edge overhead to increase in a super-linear manner.

The fundamental reason that serialization remains unavoidable in existing systems is their dependence on container-based sandboxing mechanisms. Container runtimes enforce OS-level isolation via namespaces and cgroups, completely segregating each function's address space such that even two functions co-located on the same physical node cannot exchange a single byte without routing it through an external storage intermediary [3, 4]. Prior work has focused on optimizing different components within this storage-mediated execution model. FaaSFlow [1] caches intermediate data in a local Redis instance to reduce network round-trips, while DataFlower [2] replaces pull-based storage with push-based Kafka streaming to eliminate polling overhead. However, none of these systems jointly optimize both *how* data are transferred and *where* functions are placed so as to assign the fastest available transfer path to the highest-volume communication edges.

We observe that Web Assembly (Wasm) sandboxes, unlike containers, give the runtime programmatic access to each function's

linear memory through *host functions*, i.e., callbacks invoked at the sandbox boundary for every state read or write [23]. This architectural property enables serialization-free `memmove` for co-located function pairs and direct TCP push for cross-node pairs, reducing per-edge transfer cost to the bandwidth limit of the underlying hardware. However, whether a given function pair can exploit the fast local path depends entirely on the scheduler's placement decision. Eliminating the serialization bottleneck therefore requires *co-designing* the runtime's transfer mechanisms with the scheduler's placement strategy, a cross-layer optimization that no prior system has attempted.

We present AeroFlow, a serverless workflow engine that integrates Wasm-based execution with location-aware scheduling to eliminate serialization from the inter-function data path. Specifically, AeroFlow makes three contributions.

- **Serialization-free Wasm sandbox (§3).** We propose a serialization-free Wasm sandbox that intercepts inter-function state accesses at the Wasm host-function boundary and redirects data through a per-node user-space shared-memory pool via near zero copy, eliminating serialization and external storage on every dependency edge while providing sub-millisecond sandbox cold starts.
- **Location-aware MIP scheduling (§4).** We present a location-aware MIP scheduler that formulates function placement as a constrained optimization over the workflow DAG, encoding the bandwidth gap between intra-node and cross-node network as placement-dependent edge delays to maximize the fraction of edges served by the serialization-free path.
- **A workflow engine AeroFlow (§5).** We present a transfer-policy-driven distributed workflow engine that concretizes the solver's per-edge routing decisions within the runtime system, thereby closing the scheduler–runtime co-design loop that has remained unaddressed in prior work. The engine employs a two-process architecture to reconcile the inherent conflict between the cooperative concurrency model required for DAG coordination and the parallel execution model of Wasm sandboxes.

We evaluate AeroFlow on four scientific workflow benchmarks and four real-world Python workloads deployed on an 8-node cluster. At a scale of approximately 100 functions, AeroFlow achieves a 2.4–12.4× speedup over FaaSFlow and up to 17× over DataFlower. As the scale increases to 250 functions, the relative performance advantage of AeroFlow grows to 7.8×, as container cold starts and storage contention induce super-linear performance degradation in the baseline systems. At a scale of 500 functions, AeroFlow is the only system able to complete the execution.

2 Background and Motivation

A serverless workflow composes stateless functions into a DAG where each edge encodes a data dependency. Because functions are stateless, the engine must persist every intermediate result to external storage so that each consumer can retrieve it before execution. Scientific workflows[28, 29] span diverse structures and scales: Cycles is a scientific agronomy simulation with 90 functions organized in 13 stages, where fan-out launches parallel instances producing MB-scale outputs that are aggregated in a fan-in phase.

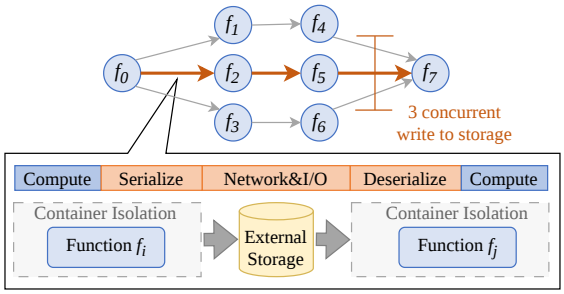
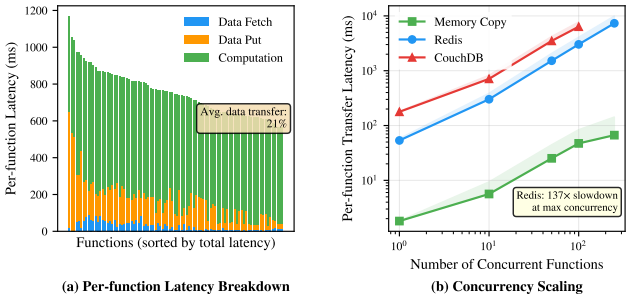


Figure 1: Traditional workflow engines introduce serialization overheads on every dependency edge.



(a) Per-function Latency Breakdown (b) Concurrency Scaling

Figure 2: The serialization bottleneck: data transfer overhead and concurrency amplification.

SoyKB (genome analysis) exhibits a similar fan-out/fan-in pattern at 98 functions with 168 dependency edges. Epigenomics and Genome represent multi-stage pipelines propagating data through successive transformation steps. In every case, every dependency edge incurs the full serialize–transfer–deserialize pipeline.

We quantify the serialization bottleneck through three controlled experiments on the Cycles workflow testing on FaaSFlow [1].

Data transfer dominates workflow latency. Figure 2(a) shows the per-function latency breakdown across all 90 functions (warm containers). Data transfer (fetch + put) accounts for 21% on average and up to 55% for aggregation functions on the critical path. Since Cycles is compute-heavy, these percentages represent a lower bound on transfer-dominated workflows.

Serialization dominates data transfer. We independently measure each cost component by transferring Python dictionaries of varying sizes (1–50 MB) through four paths: S (pure json.dumps/loads), I (Redis SET/GET on pre-serialized bytes), F (full serialize–store–retrieve–deserialize), and M (memmove baseline). Table 1 reports median latency over 20 iterations. Serialization accounts for 80–90% of Redis transfer latency; the additivity ratio $(S+I)/F$ stays within 95–102%, validating independent decomposition. Meanwhile, `memmove` is 178–323× faster than Redis. Once serialization is removed, data transfer becomes effectively free.

Concurrency amplifies storage contention. Figure 2(b) shows per-function transfer latency as N concurrent threads (1–250) each

Table 1: Four-path serialization decomposition. Serialization accounts for 80–90% of Redis transfer latency; `memmove` is 178–323× faster.

Size	S (ms)	I (ms)	F (ms)	M (ms)	Ser %	(S+I)/F
1 MB	9.2	1.5	10.7	0.03	86%	100%
5 MB	52.6	7.1	58.3	0.35	90%	102%
10 MB	110.5	17.0	128.0	0.72	86%	100%
50 MB	678.5	119.7	844.9	3.79	80%	95%

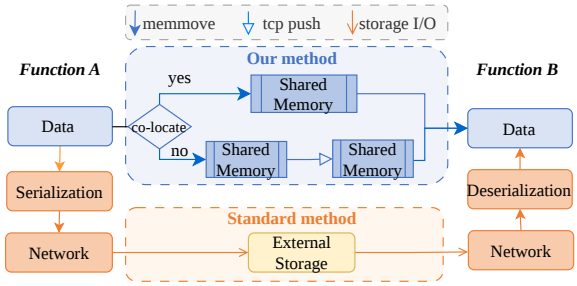


Figure 3: Data transfer path comparison. *Standard method:* 5 intermediaries, 2× serialization. *Our method:* local edges use SHM (2× `memmove`, 0× serialization); remote edges use TCP push (3× `memmove`, 0× serialization).

perform a full serialize–write–read–deserialize cycle on a 10 MB payload. Redis degrades from 53 ms (N=1) to 7,329 ms (N=250), a 137× increase that far exceeds linear scaling. The cause is Redis's single-threaded event loop, which serializes all concurrent requests. In contrast, memmove grows to only 66.7 ms (37×), as each thread operates on independent memory. This super-linear degradation is especially acute in Wasm-based runtimes where near-zero cold starts cause all functions at the same DAG level to complete simultaneously, maximizing write contention.

3 Serialization-Free Wasm Sandbox

This section addresses the mechanistic question raised in §2: *Why does serialization arise along the inter-function data path, and by what means can it be eliminated?* Figure 3 compares the relevant data paths. Our method introduces two distinct data-passing strategies that are selected based on the co-location of the communicating functions; both strategies avoid any reliance on serialization.

3.1 Why Containers Force Serialization

Serialization follows directly from the container isolation model. It becomes unavoidable when three conditions hold: C1 (Address-space isolation): each container has its own OS namespace; co-located functions cannot read each other's memory. C2 (Memory opacity): the runtime cannot inspect a function's heap layout. C3 (Non-contiguous layout): language-level structures span multiple disjoint heap allocations. Under C1–C3, moving data requires traversing the object graph (C3), copying from an opaque address space (C2), and routing through an external intermediary (C1).

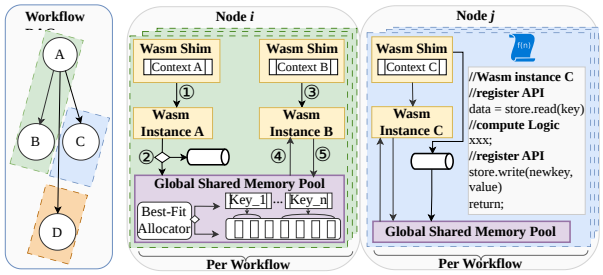


Figure 4: Intra-node execution architecture. Circled numbers trace the data path for co-located functions *A* and *B*; cross-node edges are forwarded to TCP push (§3.3).

AeroFlow replaces traditional container-based isolation with WebAssembly sandboxes that eliminate all three conditions. Each Wasm module is associated with a dedicated *linear memory* region, a contiguous, flat byte array in which all data reside at explicitly defined (offset, length) positions, thereby eliminating C3. The host runtime obtains the base address of this memory via `memory.data_ptr()` and can access any byte using standard pointer arithmetic, eliminating C1. Furthermore, every Wasm I/O operation is realized via a *host function* in the module's import namespace, which exposes the precise address and length of each data item, eliminating C2. With these three conditions removed, data movement reduces to a single `memmove` operation without traversal, format conversion, or intermediate allocation. We term this the *natural serialization* property of Wasm linear memory.

3.2 Shared Memory Pooling for Intra-Node Transfer

Having established that Wasm linear memory enables, in principle, serialization-free memory movement via 'memmove', this section presents the concrete mechanism that implements this capability in practice. Each worker node is composed of three primary components (Figure 4).

- **WasmShim.** The execution entry point for each function invocation. It maintains the function's execution context (input/output key mappings, downstream consumer lists, and data routing policy) and bridges the scheduling layer with the data transfer mechanism. Before launching the WasmInstance, the WasmShim verifies that all input keys required by the DAG are present in `Key_Dict`, ensuring data readiness before execution begins.
- **WasmInstance.** The compiled user function running inside a Wasm sandbox. It communicates with the outside world exclusively through two host functions registered in its import namespace. Both functions operate on raw pointers into the module's linear memory, and the host runtime uses the (pointer, length) pair to locate the exact byte range, enabling direct memmove without traversing any language-level object graph.
- **Global Shared Memory (SHM) Pool.** A per-node, `mmap`'d contiguous region that resides in the same process address

space as all Wasm instances yet is independent of any single instance. Two index structures manage its state: a *free list* that tracks unoccupied blocks for best-fit allocation, and a `Key_Dict` that maps each occupied key to its (offset, length) pair.

When functions A and B are co-located on Node i, the data path proceeds in five steps (Figure 4): ① the WasmShim initializes and launches WasmInstance A; ② A calls `py_write`, and the host function performs *Allocate* + *Memmove* to copy the byte range from A's linear memory into the SHM pool; ③ the WasmShim initializes WasmInstance B and verifies that all input keys are present in `Key_Dict`; ④ B is launched and calls `py_read`; the host function looks up the key in `Key_Dict`, locates the corresponding slot in the SHM pool, and copies the data into B's linear memory via a single `memmove`; ⑤ B completes and writes its own output back to the pool via `py_write`. For edges whose consumer resides on a different node (e.g., $A \rightarrow C$), step ② additionally triggers the cross-node push described in §3.3. Across the entire local write–read cycle, data traverses exactly two memmove operations and zero serialization steps.

The preceding walkthrough traced the end-to-end path through Figure 4. We now detail the primitive SHM operations shown in Figure 5: part (a) illustrates the write and read primitives; part (b) illustrates memory reclamation. These operations address the concurrency and capacity challenges identified in §3.1.

Three-Phase Write Protocol. When the MIP scheduler co-locates 20–30 functions on a single node, multiple producers at the same DAG level may call `py_write` simultaneously. A naive design that holds the pool lock for the entire memmove would block all other writers for the duration of the copy (milliseconds at 30–50 MB), serializing concurrent producers and negating the co-location benefit. The three-phase protocol eliminates this bottleneck by splitting the write into three stages with different lock requirements (Figure 5(a), *Allocate–Memmove–Publish*):*Phase 1—Allocate.* The writer acquires the pool lock, selects a best-fit block from the free list, removes it to reserve the slot, and releases the lock. Only free-list metadata is updated, so this phase completes in microseconds.*Phase 2—Memmove.* The writer copies raw bytes from the producer's linear memory into the reserved slot. Because the slot is exclusively owned, this step executes *outside* the lock and releases the Python GIL, enabling true parallelism among concurrent large transfers.*Phase 3—Publish.* The writer re-acquires the lock, registers the key in `Key_Dict` with its (offset, length) pair, and releases the lock. Only after this point is the data visible to consumers. This final metadata update again completes in microseconds.

By confining mutual exclusion to two microsecond-scale metadata phases and excluding the millisecond-scale bulk copy, the protocol allows N concurrent writers to overlap their memmove phases with near-perfect parallelism.

Lock-Free Read. If reads also acquired the pool lock, each `py_read` would serialize against concurrent writers, defeating the purpose of co-location. The three-phase write protocol makes reads inherently safe without any lock. As shown in Figure 5(a) (*Prefetch*), when a consumer calls `py_read`, the host function looks up `Key_Dict` via a single `dict.get` (atomic under CPython's GIL) and executes a `memmove` from the SHM slot directly into the consumer's linear memory. No lock is acquired at any point. Lock-free reads are safe for two reasons: (1) the three-phase write publishes the key only *after* the data has been fully written (Phase 3 follows Phase 2), so a reader that finds a key is guaranteed to see complete data; (2) the DAG's precedence constraint ensures that a consumer is scheduled only after all its producers have completed, so the key is always present when the consumer executes.

Memory Reclamation. The SHM pool has finite capacity, yet a workflow's aggregate intermediate data can exceed it by an order of magnitude. Figure 5(b) illustrates the reclamation strategy. Once a consumer has read a key, the runtime immediately *deletes* the entry from `Key_Dict` and returns the block to the free list. Because the DAG guarantees each key has exactly one consumer, no reference counting is required. To combat fragmentation from interleaved allocations and deallocations, the runtime *merges* adjacent free-list entries upon each deallocation, consolidating scattered fragments into contiguous blocks. This produces a pipeline effect in which each DAG level's outputs are consumed and freed before the next level begins, bounding peak occupancy by the widest single DAG level rather than total data volume.

3.3 Cross-Node Transfer via TCP Direct Push

For cross-node edges (e.g., $A \rightarrow C$ where C resides on Node j), the runtime extends the serialization-free guarantee via sender-driven TCP direct push.

Routing-map-driven branching. The MIP scheduler (§4) computes a per-function routing map that classifies each output key as Local or Remote. When A calls `py_write`, the runtime inspects this map and acts accordingly. Local keys remain in the pool; remote keys are read from the SHM slot and pushed over persistent TCP connections to the destination node's listener.

Serialization-free guarantee. The bytes pushed over TCP are the same contiguous sequence deposited by `py_write`, with no parsing, format conversion, or schema envelope applied. The only framing is a fixed-length binary header (request ID, key length, payload length) for demultiplexing. On the destination node, the TCP listener writes the payload directly into its local SHM pool via memmove.

Unified read path. From this point, data is indistinguishable from locally produced data. When C calls `py_read`, it reads from its own node's SHM pool through the same lock-free Prefetch path, entirely unaware of whether the input arrived locally or over the network. This design decouples placement decisions from runtime code paths, allowing the scheduler to reassign functions between nodes without modifying any transfer logic. Across both paths, the local path requires two memmove calls and the remote path requires three; no serialization format is applied at any stage.

4 Location-Aware MIP Scheduling

§3 established two data paths with different performance: intra-node shared-memory transfer usually completes faster than cross-node TCP push. Which path a given dependency edge takes is entirely determined by the scheduler's placement decision. A placement-unaware scheduler (e.g., greedy bin-packing) optimizes load balance but ignores data volumes, scattering data-heavy edges across node boundaries. Wasm's near-zero cold starts amplify this issue

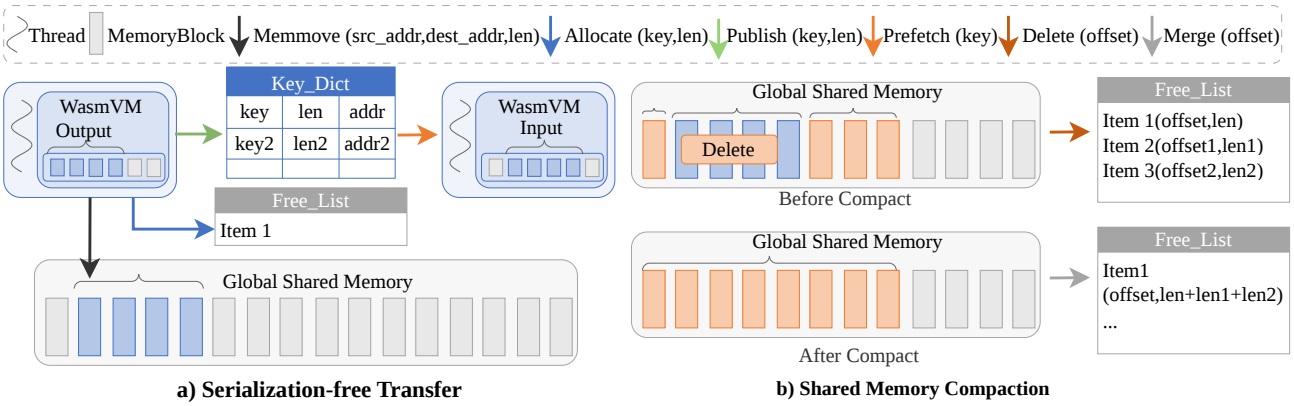


Figure 5: Shared memory operations. (a) Serialization-free transfer: *Allocate* reserves a free-list slot; *Memmove* copies raw bytes from the producer's linear memory; *Publish* registers the key in `Key_Dict`; *Prefetch* reads from the pool into the consumer's linear memory. (b) Memory reclamation: consumed keys are deleted and adjacent free blocks are coalesced.

because all functions at the same DAG level become ready simultaneously, maximizing concurrent cross-node transfers and network contention (§2). AeroFlow therefore co-designs placement with the runtime mechanisms of §3, formulating function assignment as a mixed-integer program (MIP) whose objective is to minimize end-to-end workflow makespan by co-locating data-heavy edges onto the fast shared-memory path.

4.1 Problem Formulation

We model a serverless workflow as a weighted DAG $G = (V, E)$, where each vertex $f \in V$ is a function with known execution time r_f (profiled offline), and each directed edge $e = (u, v) \in E$ carries a data dependency whose transfer time equals d_e when sent over the network. When both endpoints reside on the same node, the transfer uses the shared-memory path and the delay reduces to d_e/α, where $\alpha = c_R/c_L$ is the ratio of remote to local per-edge transfer cost. Here c_R denotes the unit transfer cost on the remote path (e.g., serialization plus network I/O through Redis or CouchDB) and c_L denotes the unit cost on the local shared-memory path. The concrete value of α depends on the runtime and storage backend: for storage-mediated systems such as FaaSFlow with Redis/CouchDB, $\alpha \approx 50$–100; for AeroFlow's TCP direct push versus SHM path, $\alpha \approx 5$–10. Treating α as a tunable parameter makes the formulation applicable to any combination of workflow engine and function runtime. The infrastructure provides $|N|$ worker nodes, each with concurrency capacity C_n (the maximum number of concurrent execution slots). Every function f declares a *scale* value s_f, specifying how many slots it occupies when running.

The placement problem is formulated as a *Constrained Mixed-Integer Program (MIP)*. For each function f and node n, binary variable $x_{f,n} = 1$ indicates f is assigned to n. Continuous variable $t_f \geq 0$ denotes the earliest start time of f, and continuous variable T denotes the makespan. For each edge $e = (u, v)$ and node n, auxiliary binary variable $y_{e,n}$ indicates co-location of both endpoints on n; the *locality indicator* $\ell_e = \sum_n y_{e,n}$ equals 1 when both endpoints share a node and 0 otherwise. The objective is to minimize the workflow makespan subject to capacity and precedence constraints, explicitly modeling the bandwidth gap between the intra-node shared-memory path and the cross-node network path as placement-dependent edge delays:

$$\min \quad T \tag{1}$$

$$\text{s.t.} \quad \sum_{n\in N} x_{f,n} = 1, \quad \forall f \in V \tag{C1}$$

$$\sum_{f\in V} s_f \cdot x_{f,n} \leq C_n, \quad \forall n \in N \tag{C2}$$

$$y_{e,n} \leq x_{u,n}, \;\; y_{e,n} \leq x_{v,n}, \;\; y_{e,n} \geq x_{u,n}+x_{v,n}-1, \quad \forall e=(u,v)\in E, \; n\in N \tag{C3}$$

$$t_v \geq t_u + r_u + d_e, \quad \forall (u,v) \in E \tag{C4a}$$

$$t_v \geq t_u + r_u + d_e/\alpha - M(1-y_{e,n}), \quad \forall (u,v)\in E, \; n\in N \tag{C4b}$$

$$T \geq t_f + r_f, \quad \forall f \in V \tag{C5}$$

$$x_{f,n} \in \{0,1\}, \; y_{e,n} \in \{0,1\}, \; t_f \geq 0 \tag{C6}$$

Objective and Constraints. The objective minimizes the workflow makespan T, defined as the completion time along the critical (longest) path through the DAG. Because execution times r_f are fixed, the only lever available to the scheduler is the transfer delay on each edge, which is controlled entirely by the placement decision.

C1 ensures each function is assigned to exactly one node. C2 enforces per-node capacity: the total scale demand $\sum_f s_f \cdot x_{f,n}$ must not exceed the concurrency limit C_n. C3 linearizes the co-location indicator $y_{e,n} = x_{u,n} \wedge x_{v,n}$ via standard McCormick envelopes, enabling explicit modeling of intra-node data locality without introducing quadratic terms.

C4a and C4b jointly encode DAG precedence with a placement-dependent transfer delay. Let δ_e denote the effective transfer delay on edge e: $\delta_e = d_e$ when the two endpoints reside on different nodes (remote path), and $\delta_e = d_e/\alpha$ when they are co-located (shared-memory path). C4a conservatively assumes the remote case for every edge; C4b tightens the delay to the local value via

a big-M relaxation that activates only when both endpoints are co-located ($y_{e,n} = 1$). Together they enforce the timing model: a successor cannot start until its predecessor has completed and data has arrived, with the effective delay determined by the placement decision. M is set to the sum of all execution times and edge weights to guarantee a valid relaxation.

C5 bounds the makespan as the latest completion time among all functions. C6 defines the variable domains. Together with the objective, the solver jointly decides placement ($x_{f,n}$) and timing (t_f), co-locating data-heavy edges onto the fast shared-memory path to shorten the critical path. The formulation produces $O\big((|V|+|E|)\,|N|\big)$ variables and constraints; the direct linearization in C3 avoids the quadratic $O(|E|\,|N|^2)$ expansion that a naive indicator-constraint formulation would require.

4.2 Solution Strategy and Runtime Integration

The placement problem is NP-hard (reducible to minimum-k-cut). We use the SCIP solver [24] to solve the MIP formulation with a tractable configuration: parallel branch-and-bound, a 5% MIP gap tolerance to accept near-optimal solutions early, aggressive presolving and root-node cutting planes to tighten relaxations.

After solving, we extract the per-function node assignments $x^*_{f,n}$ and start times t^*_f, then recover the critical path to identify bottleneck edges and aid debugging. For workflows exceeding the time limit, we fall back to a greedy critical-path heuristic. Functions are first assigned via hash-based bin-packing, then topological-order DP computes start times, and the heuristic iteratively merges the heaviest cross-node edge on the critical path until no further co-location is feasible under capacity constraints. The heuristic runs in sub-second time but yields a lower co-location ratio than the exact MIP solution.

The solver output is agnostic to the underlying runtime and directly consumable by any system supporting per-node function placement and path selection between local/remote dependencies. In AeroFlow, solved assignments populate distributed scheduling tables to direct function dispatch, while co-location variables label each edge as local or remote. This closes the co-design loop, ensuring placement and data paths are jointly optimized rather than independently configured.

5 AeroFlow Workflow Engine

This section describes the workflow engine that integrates the aforementioned sandbox mechanism and scheduling approach into a coherent system. The central design challenge is unique to lightweight-sandbox systems. Because Wasm functions start in under one millisecond, the engine loses the implicit temporal buffer that container cold-start latency provides and must compensate through explicit runtime mechanisms.

5.1 Architecture Overview

Figure 6 shows the end-to-end architecture. The MIP scheduler produces two artifacts: (i) function placement, i.e., the node assignment $x^*_{f,n}$, and (ii) a per-function routing map labeling each outgoing edge as Local or Remote. These labels are materialized as the routing map (§3.3), closing the co-design loop between scheduler and runtime. Each worker node runs two cooperating processes:

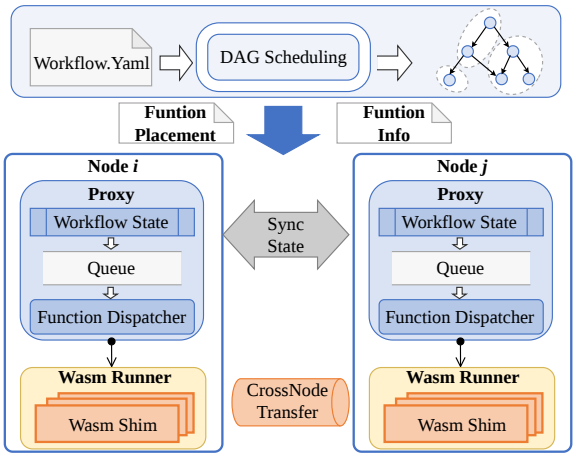


Figure 6: AeroFlow's architecture.

[topsep=2pt,itemsep=1pt]

- **Proxy.** A coroutine-based server that tracks per-request workflow state and dispatches ready functions. Nodes independently advance the DAG and synchronize completion events with peer proxies, requiring no central coordinator.
- **Wasm Runner.** A native-threaded daemon owning the Wasm engine, the SHM pool, and a thread pool for parallel sandbox execution.

The two-process split resolves a concurrency-model mismatch. The proxy's cooperative coroutines conflict with the Wasm runtime's FFI requirement for real OS threads and OS-level locks; colocating both in one process causes deadlocks when host functions acquire locks outside the coroutine scheduler. The process boundary provides concurrency-semantic isolation whereby the proxy multiplexes thousands of workflow states cooperatively, while the runner exploits true thread-level parallelism.

After a function completes, the runner consults the routing map. Local successors read output directly from the SHM pool; for each remote successor, the runner pushes raw bytes via TCP to the destination node's pool. Every consumer reads from its local pool regardless of data origin, so the routing map translates placement decisions directly into runtime transfer paths.

5.2 Flow Control

In container-based engines, cold-start latency (0.5–2 s) provides implicit temporal padding and upstream data arrives long before the container is ready. AeroFlow's near-zero startup (<1 ms) removes this guarantee, so a function dispatched immediately after predecessors complete may begin before cross-node data has landed.

AeroFlow replaces this implicit buffering with two explicit mechanisms. First, *data-ready dispatch*: each proxy maintains two bitmaps per pending function, B_c (predecessor completion) and B_d (data presence in local SHM). For local predecessors, both bits set atomically on completion; for remote predecessors, B_c sets on

Table 2: Systems under evaluation.

System	Scheduler	Sandbox	Data Path
AeroFlow	MIP	Wasm	SHM + TCP
FaaSFlow [1]	Greedy	Docker	Redis/CouchDB
DataFlower [2]	Master	Docker	Redis/Kafka
AeroFlow-Greedy	Greedy	Wasm	SHM + TCP
AeroFlow-Container	MIP	Docker	Redis/CouchDB

peer-proxy notification and B_d sets when the TCP receiver confirms the data is written. A function dispatches only when $B_c(f) = \mathbf{1} \wedge B_d(f) = \mathbf{1}$, naturally staggering activations across each DAG level.

Second, *admission-controlled fetch*: when multiple co-located functions become dispatch-ready simultaneously, they all enter the fetch phase, copying input from SHM into Wasm linear memory. To prevent memory-bandwidth saturation, the Wasm Runner gates fetch entry with a semaphore of size $|S| = \lfloor B_{\text{mem}}/\bar{d} \rfloor$, where B_{mem} is measured memory bandwidth and $\bar{d}$ is average per-function input size. Only the memory-intensive copy phase is rate-limited; functions that have completed fetch execute in full parallelism.

5.3 Memory Lifecycle

The SHM pool must support workflows whose aggregate intermediate data far exceeds its capacity. AeroFlow uses a *consume-and-evict* lifecycle: once a function fetches all inputs, consumed entries are immediately deleted and returned to the free list. For deep DAGs, early-stage outputs are reclaimed before late-stage functions produce new data, so peak occupancy is bounded by the widest single DAG level rather than total data volume. Combined with the three-phase write protocol (§3.2) and free-list coalescing, this enables workflows whose cumulative intermediate data exceeds pool capacity by an order of magnitude.

6 Evaluation

We evaluate AeroFlow to answer four questions:

- **Q1** How does AeroFlow's serialization-free data path compare to storage-mediated systems as payload size grows? (§6.2)
- **Q2** How does AeroFlow scale with increasing function count? (§6.3)
- **Q3** How much does each core optimization (MIP scheduling and Wasm+SHM) contribute? (§6.4)
- **Q4** Can AeroFlow execute real-world Python workloads, and how does it compare to FaaSFlow on applications with diverse computation profiles? (§6.5)

6.1 Experimental Setup

Cluster. We deploy all systems on a cluster of 8 nodes (7 workers + 1 master), each with an Intel(R) Xeon(R) 6982P-C (8 cores / 16 threads), 16 GB DDR4 RAM, connected via 8 Gbps Ethernet, running Ubuntu 22.04 LTS.

Baselines. We compare five system configurations (Table 2). AeroFlow-Greedy and AeroFlow-Container are ablation variants that isolate the contributions of MIP scheduling and the Wasm+SHM runtime, respectively.

Workloads. We use four synthetic scientific workflow benchmarks with configurable scale (Table 3). Each function executes a configurable sleep (default 0.5 s) to simulate computation and produces a configurable-size output.

Methodology. We vary per-function output size across 5, 10, 20, and 50 MB. Each experiment uses closed-loop testing (serial submission). Per-edge transfer latency is computed as the total data read time plus write time divided by the number of DAG edges, decomposed into serialization (de/serialization) and pure I/O components.

6.2 Data Transfer vs. Payload Size

Figure 7 shows how data transfer overhead scales with per-function output size (5–50 MB) across four workflows.

6.2.1 End-to-End Latency. . AeroFlow achieves 2.4–12.4× speedup over FaaSFlow across all configurations, with the advantage widening at larger payloads. The mechanism is straightforward: AeroFlow's SHM `memmove` scales linearly with data size, whereas FaaSFlow's JSON serialization grows super-linearly due to recursive object-graph traversal. On fan-out workflows (Cycles, SoyKB), Redis write contention further amplifies the gap as the single-threaded event loop serializes 40+ concurrent SET operations. Counter-intuitively, FaaSFlow's E2E latency on SoyKB and Cycles is nearly flat across data sizes (~32–36 s for 5–50 MB). The reason is that Docker cold starts (~90 functions × 0.5–1 s each) dominate the critical path to the point where incremental transfer cost is masked: even at 50 MB, the additional serialization time per edge (~300 ms) is small relative to the accumulated container startup along a 9–13 stage DAG (>25 s). AeroFlow eliminates both bottlenecks simultaneously, making its E2E sensitive only to actual data volume. DataFlower's centralized master creates a topology-dependent bottleneck: AeroFlow achieves 10–18× speedup on fan-in/out workflows (SoyKB) but only 1.5–3.1× on pipeline-structured Genome where the master is not contended.

6.2.2 Per-Edge Breakdown. . AeroFlow and AeroFlow-Greedy show *zero* serialization overhead at all data sizes, while FaaSFlow and DataFlower incur 35–299 ms per edge at 10–50 MB for JSON/protobuf encoding. After removing serialization, AeroFlow's residual I/O cost at 10 MB ranges from 33.4 ms (Epigenomics) to 81.8 ms (Genome). The gap reflects DAG topology: Epigenomics has 32–96 concurrent functions per stage, so many edges transfer in parallel and the per-edge average is amortized across the wide level; Genome's pipeline structure forces edges onto the sequential critical path with no such amortization. FaaSFlow's pure I/O cost (excluding serialization) is up to 25× higher on pipeline-structured Genome, where CouchDB's MVCC conflict resolution serializes sequential writes along the critical path.

6.3 Scalability with DAG Size

Figure 8 evaluates scaling from 50 to 500 functions at fixed 10 MB output. AeroFlow's latency grows moderately up to 250 functions (SoyKB: 6.3 s→8.5 s) and completes all four workflows at 500 functions in 33.9–35.6 s. FaaSFlow degrades super-linearly and times out beyond 100–250 functions (SoyKB: 15.8 s→66.4 s) as Docker cold

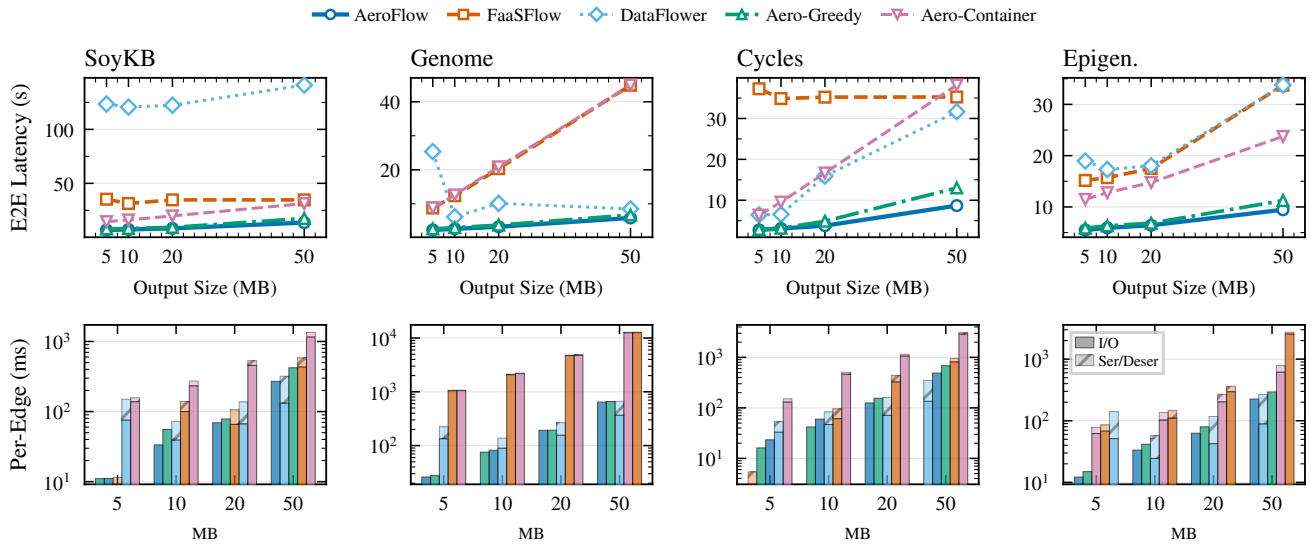


Figure 7: Data transfer performance vs. per-function output size (5–50 MB), one column per workflow. Row 1: End-to-end workflow latency for all five systems. Row 2: Per-edge transfer latency (log scale) decomposed into pure I/O (solid) and serialization (hatched). AeroFlow eliminates serialization entirely.

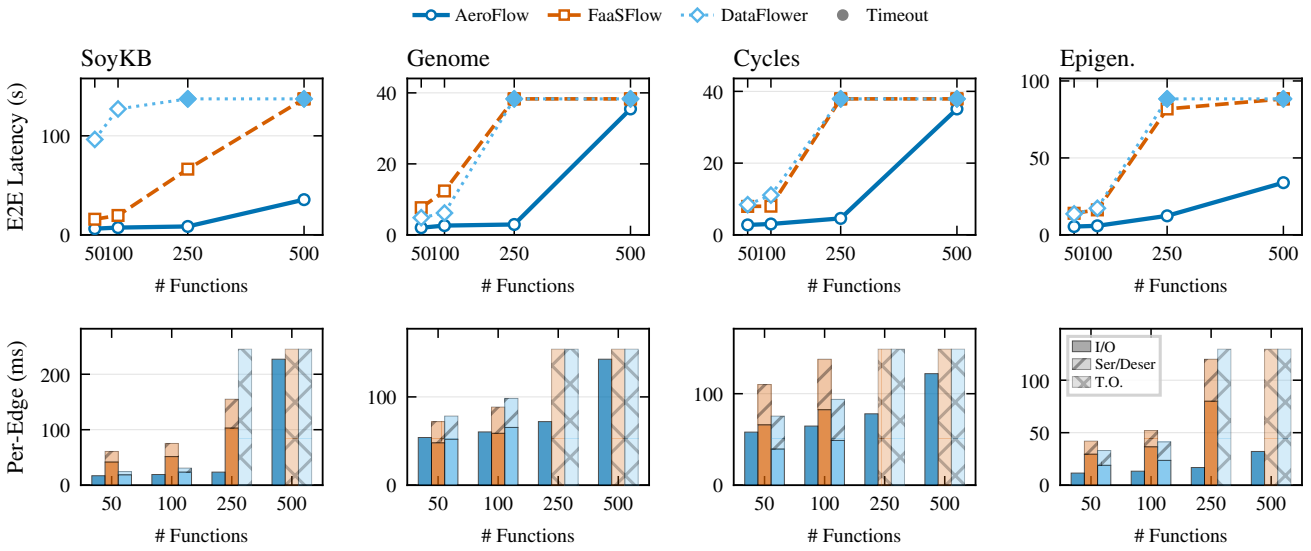


Figure 8: Scalability analysis: performance vs. function count (50–500), 10 MB output, 0.5 s compute. Row 1: E2E latency (solid = AeroFlow, dashed = FaaSFlow, dotted = DataFlower; filled marker = timeout). Row 2: Per-edge transfer breakdown (stacked: I/O + serialization); cross-hatched bars denote timeouts.

starts and Redis contention accumulate; DataFlower similarly saturates beyond 100 functions. At 250 functions, AeroFlow's speedup reaches 7.8× (SoyKB); at 500 functions it is the only system that completes. Per-edge latency remains 1.5–3.9× lower than FaaSFlow at 100 functions; at 500 functions it rises to 32–227 ms, as MIP scheduling co-locates 75–89% of edges for sub-millisecond SHM transfer (Figure 9h).

6.4 Ablation Study

Figure 9 (Row 1) isolates each optimization's contribution across four data sizes per workflow.

6.4.1 Wasm+SHM runtime (dominant factor). Replacing Wasm+SHM with Docker+Redis (AeroFlow-Container) increases latency by 2.1–7.7×. The penalty grows with data size (Genome: 3.8× at 5 MB → 7.7× at 50 MB) because serialization cost is super-linear while `memmove` is linear, and pipeline structures compound this per-edge

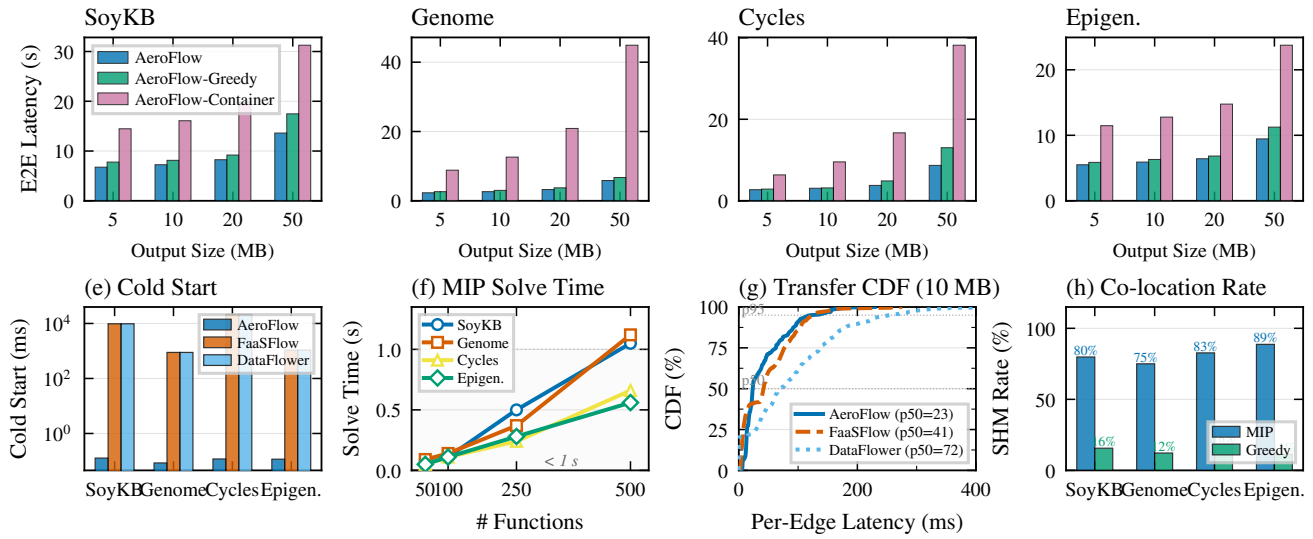


Figure 9: Ablation and supporting analysis. Row 1: Ablation study per workflow—AeroFlow vs. AeroFlow-Greedy (−MIP) vs. AeroFlow-Container (−Wasm+SHM) across data sizes. Row 2: (e) Cold start comparison; (f) MIP solve time vs. function count; (g) CDF of per-edge latency at 10 MB; (h) SHM co-location rate.

Table 3: Workflow benchmark characteristics

Workflow	Funcs	DAG Edges	Structure	Data Pattern
Genome	100	100	Pipeline	Heavy chains
SoyKB	98	168	Fan-in/out	Locality-sensitive
Epigenomics	91	107	Staged	Multi-stage sync
Cycles	90	210	Nested fan-out	Wide parallelism

Table 4: Real-world Python workloads: AeroFlow vs. FaaSFlow E2E latency (seconds, 5-run mean ± std).

Workload	AeroFlow	FaaSFlow	Speedup
WordCount	3.42±0.18	5.67±0.43	1.7×
FileProc	2.68±0.11	4.35±0.28	1.6×
IllgalRecog	8.15±0.92	24.87±1.53	3.1×
Scientific	4.83±0.15	5.94±0.38	1.2×

cost across 100 sequential dependencies. Fan-out workflows show smaller penalties (2.1–4.4×) because wider parallelism amortizes cold-start overhead across concurrent branches.

6.4.2 MIP scheduling (complementary factor). Replacing MIP with greedy placement (AeroFlow-Greedy) increases latency by 1.03–1.50×, with the benefit most pronounced at large data sizes where the SHM-vs-TCP gap matters most (Cycles at 50 MB: 1.50×).

The two optimizations are complementary: the serialization-free runtime eliminates the dominant per-edge cost, while MIP scheduling reduces the residual SHM-vs-TCP cost by maximizing co-location. Neither alone achieves AeroFlow's full performance.

6.5 Micro-Benchmarks and Real-World Workloads

Figure 9 (Row 2) provides supporting micro-benchmarks. Wasm cold start is ~0.1 ms vs. Docker's 560–1,273 ms (Figure 9e). SCIP solves 100-function workflows in <0.15 s and 500-function workflows in <1.2 s (Figure 9f), negligible relative to execution time. AeroFlow's per-edge p50/p95 is 23/121 ms vs. FaaSFlow's 41/125 ms and DataFlower's 72/253 ms (Figure 9g); the tighter distribution reflects deterministic SHM `memmove`. MIP achieves 75–89% co-location vs. 11–18% for greedy (Figure 9h).

We extend the Wasm runtime with a CPython interpreter compiled to WebAssembly, enabling execution of unmodified Python code without source-code modification. Table 4 compares AeroFlow against FaaSFlow on four Python workflows.

AeroFlow achieves 1.2–3.1× speedup. IllgalRecog benefits most (3.1×) because FaaSFlow reloads the full ML runtime on every cold invocation across multiple stages, whereas AeroFlow amortizes this into a single pre-compiled binary. Compute-light workloads (WordCount, FileProc: 1.6–1.7×) gain primarily from cold-start elimination; compute-heavy Scientific (1.2×) leaves less room for infrastructure-level optimization.

7 Discussion

Applicability and Data Contract. AeroFlow targets acyclic workflows with known dependencies. Its serialization-free fast path applies only to intermediate values already materialized as a shared contiguous byte buffer: (pointer, length) names a transport range in Wasm linear memory, not a cross-language object ABI. Python bytes/bytearray can use this path; dict, list, and application/json values must be json-encoded by an adapter. Heterogeneous-language functions need an agreed buffer format; arbitrary native objects and dynamic or cyclic graphs are out of scope of this paper.

Generality and Related Designs. Wasm does not unify language-specific object layouts. It provides a portable sandbox, bounds-checked host-visible linear memory, and a stable host-function boundary. Prior systems apply related optimizations via shared processes [14], elastic or cached state services [15, 16], OS/runtime remote mapping with RDMA [13], or a LibOS single address space [12]. AeroFlow combines per-function Wasm isolation, SHM/TCP byte-buffer transfer, and placement-aware routing.

Runtime Overhead. Runtime transfer cost is determined primarily by memory bandwidth for intra-node (local) communication edges and by TCP bandwidth and framing overhead for inter-node (remote) communication edges. Additional sources of overhead include reserved shared-memory (SHM) capacity, offline profiling, and MIP solving. The complexity associated with larger or dynamically changing deployments could be mitigated through decentralized or hierarchical scheduling strategies, analogous to those employed in Wukong [17].

Fault Tolerance. The three-phase write protocol and data-ready dispatch mechanism together ensure failure-free consistency: a key is made visible to readers only after its replica has been fully written, and a consumer is instantiated only after all of its input data have arrived. However, these mechanisms do not provide crash durability. Both shared memory (SHM) and in-flight TCP state are volatile, and the current prototype lacks durable logging and replay, application-level checksums or acknowledgements, node-crash recovery mechanisms, and systematic fault-injection–based evaluation. Transactional and shared-log architectures [18, 19] may therefore serve as complementary approaches to augment AeroFlow with strong durability and recovery guarantees.

8 Related Work

Serverless workflow systems. FaaSFlow [1] introduced worker-side scheduling with local Redis caching; DataFlower [2] replaces polling with Kafka streaming. Both retain full serialization on every edge. Quilt [8] merges functions at LLVM IR level; Metis [9] proposes OS-level workflow-aware scheduling via eBPF; FUYAO [10] uses DPU hardware to accelerate inter-function state transfer. None jointly optimizes placement with the data transfer mechanism.

Wasm-based serverless. Faasm [5] pioneered Wasm faaslets with shared-memory regions but falls back to Redis for cross-node transfers and lacks workflow-aware scheduling. Featherlight [6] achieves lock-free zero-copy for single-node ML inference pipelines. Roadrunner [7] introduces a sidecar shim achieving 97% serialization reduction via three communication modes (user-space, kernel-space mmap, and network), but as a pluggable transport it does not influence placement. AeroFlow differs in three respects: (1) a managed key-value pool enabling user-transparent exchange without API modification; (2) pure user-space memmove without kernel calls; (3) co-design with the MIP scheduler ensuring data-heavy edges use the fastest path. Jord [11] and AlloyStack [12] achieve zero-copy via single-address-space or library-OS designs with hardware support (Intel MPK), targeting stronger isolation guarantees complementary to our approach.

Cold start and resource management. Firecracker [4] reduces VM startup via microVMs; AFaaS [3] and Incendio [21] optimize cold start within the container paradigm. Leopard [20] proposes decoupled CPU/memory billing; ServerlessLLM [22] optimizes LLM inference scheduling. AeroFlow sidesteps cold start entirely via Wasm sandboxes (<0.1 ms).

Workflow-aware scheduling and data passing. Recent studies have explored optimizing function placement and data passing for serverless workflows. SONIC [25] dynamically selects the optimal data-passing method (e.g., VM-storage, direct-passing) for each DAG edge and implements communication-aware function placement. WiseFuse [26] proposes DAG transformations, such as function fusion and bundling, to generate optimized execution plans that reduce end-to-end latency. FaaSPR [27] introduces a latency-oriented placement and routing scheduler that groups instances with data dependencies to minimize cross-server transmission. While these systems optimize placement or routing, they still rely on traditional serialization or container-based data exchange. AeroFlow distinguishes itself by co-designing a serialization-free Wasm shared-memory runtime with a global MIP scheduler, simultaneously eliminating serialization overhead and maximizing the use of the fast local path.

9 Conclusion

Serverless computing has emerged as a widely adopted execution paradigm for constructing complex applications from fine-grained, stateless functions orchestrated as workflows. This paper presents AeroFlow, a serverless workflow engine that co-designs WebAssembly (Wasm)-based execution with location-aware mixed-integer programming (MIP) scheduling to eliminate serialization along the inter-function data path. By replacing containers with Wasm sandboxes whose linear-memory model enables direct byte-level access, AeroFlow enables serialization-free state transfer via shared-memory `memmove`, thereby reducing per-edge communication latency to the bandwidth limit of the underlying hardware. The MIP-based scheduler maximizes the co-location of data-intensive function pairs, ensuring that the lowest-latency transfer path is systematically applied to the highest-volume edges. This design cuts cold-start overhead from seconds to 0.1 ms and enables microsecond-scale intra-node data exchange. These techniques yield 2.4–12.4× end-to-end speedups over FaaSFlow and up to 17× over DataFlower at scales of about 100 functions. At 500 functions, baseline systems time out, while AeroFlow executes workflows successfully. AeroFlow shows that jointly optimizing the runtime data path and scheduling policy achieves performance gains unattainable by either optimization alone.

Acknowledgment

We would very much like to thank the anonymous reviewers for their valuable comments. Special thanks must go to the overall RAIDS Lab team at Beihang University, for their collaborative contribution and countless technical discussion. This work is supported in part by National Key R&D Program of China (Grant No. 2024YFB4505901), in part by the National Natural Science Foundation of China (Grant No. 62402024, 62522201), in part by Beijing Natural Science Foundation (No. L241050), in part by the Fundamental Research Funds for the Central Universities, and is supported by project 2023WDZC01002. For any correspondence, please refer to the project lead Renyu Yang (renyuyang@buaa.edu.cn).

References

[1] Zijun Li, Yushi Liu, Linsong Guo, Quan Chen, Jiagan Cheng, Wenli Zheng, and Minyi Guo. 2022. FaaSFlow: Enable Efficient Workflow Execution for Function-as-a-Service. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*, February 28 - March 4, 2022, Lausanne, Switzerland. ACM, New York, NY, USA, 1220–1235. https://doi.org/10.1145/3503222.3507717

[2] Zijun Li, Chuhao Xu, Quan Chen, Jieru Zhao, Chen Chen, and Minyi Guo. 2024. DataFlower: Exploiting the Data-flow Paradigm for Serverless Workflow Orchestration. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, April 27 - May 1, 2024, La Jolla, CA, USA. ACM, New York, NY, USA, 81–95. https://doi.org/10.1145/3623278.3624755

[3] Xiaohu Chai, Tianyu Zhou, Keyang Hu, Jianfeng Tan, Tiwei Bie, Anqi Shen, Dawei Shen, Qi Xing, Shun Song, Tongkai Yang, Le Gao, Feng Yu, Zhengyu He, Dong Du, Yubin Xia, Kang Chen, and Yu Chen. 2025. Fork in the Road: Reflections and Optimizations for Cold Start Latency in Production Serverless Systems. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*, July 7 - 9, 2025, Santa Clara, CA, USA. USENIX Association, Berkeley, CA, USA, 1–21.

[4] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*, February 25 - 27, 2020, Santa Clara, CA, USA. USENIX Association, Berkeley, CA, USA, 419–434.

[5] Simon Shillaker and Peter Pietzuch. 2020. Faasm: Lightweight Isolation for Efficient Stateful Serverless Computing. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*, July 15 - 17, 2020. USENIX Association, Berkeley, CA, USA, 419–433.

[6] Xingguo Pang, Liu Liu, Yanze Zhang, Zhuofu Chen, Zhijun Ding, Dazhao Cheng, and Xiaobo Zhou. 2025. Featherlight Stateful WebAssembly for Serverless Inference Workflows. *IEEE Transactions on Parallel and Distributed Systems* 36, 6 (2025), 1651–1665. https://doi.org/10.1109/TPDS.2025.3530335

[7] Cynthia Marcelino, Thomas Pusztai, and Stefan Nastic. 2025. Roadrunner: Accelerating Data Delivery to WebAssembly-Based Serverless Functions. In *Proceedings of the 26th International Middleware Conference (MIDDLEWARE '25)*, December 8 - 12, 2025, Madrid, France. ACM, New York, NY, USA. https://doi.org/10.1145/3721462.3770777

[8] Yuxuan Zhang and Sebastian Angel. 2025. Quilt: Resource-aware Merging of Serverless Workflows. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP '25)*, October 13 - 16, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 21 pages. https://doi.org/10.1145/3731569.3764830

[9] Wenda Tang, Yanan Yang, and Jie Wu. 2025. Metis: A Non-Clairvoyant, Workflow-Aware OS Scheduler for Serverless Applications. In *Proceedings of the 16th ACM Symposium on Cloud Computing (SoCC '25)*, November 20 - 22, 2025. ACM, New York, NY, USA.

[10] Guowei Liu, Laiping Zhao, Yiming Li, Zhaolin Duan, Sheng Chen, Yitao Hu, Zhiyuan Su, and Wenyu Qu. 2024. FUYAO: DPU-enabled Direct Data Transfer for Serverless Computing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, April 27 - May 1, 2024, La Jolla, CA, USA. ACM, New York, NY, USA. https://doi.org/10.1145/3620666.3651327

[11] Yuanlong Li, Atri Bhattacharyya, Madhur Kumar, Abhishek Bhattacharjee, Yoav Etsion, Babak Falsafi, Sanidhya Kashyap, and Mathias Payer. 2025. Single-Address-Space FaaS with Jord. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*, June 21 - 25, 2025, Tokyo, Japan. IEEE/ACM.

[12] Jianing You, Kang Chen, Laiping Zhao, Yiming Li, Yichi Chen, Yuxuan Du, Yanjie Wang, Luhang Wen, Keyang Hu, and Keqiu Li. 2025. AlloyStack: A Library Operating System for Serverless Workflow Applications. In *Proceedings of the 20th European Conference on Computer Systems (EuroSys '25)*, March 30 - April 3, 2025, Rotterdam, Netherlands. ACM, New York, NY, USA. https://doi.org/10.1145/3689031.3717490

[13] Fangming Lu, Xingda Wei, Zhuobin Huang, Rong Chen, Minyu Wu, and Haibo Chen. 2024. Serialization/Deserialization-free State Transfer in Serverless Workflows. In *Proceedings of the 19th European Conference on Computer Systems (EuroSys '24)*, 132–147. https://doi.org/10.1145/3627703.3629568

[14] Swaroop Kotni, Ajay Nayak, Vinod Ganapathy, and Arkaprava Basu. 2021. Faastlane: Accelerating Function-as-a-Service Workflows. In *2021 USENIX Annual Technical Conference (USENIX ATC '21)*, 805–820. https://www.usenix.org/conference/atc21/presentation/kotni

[15] Ana Klimovic, Yawen Wang, Patrick Stuedi, Animesh Trivedi, Jonas Pfefferle, and Christos Kozyrakis. 2018. Pocket: Elastic Ephemeral Storage for Serverless Analytics. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18)*, 427–444. https://www.usenix.org/conference/osdi18/presentation/klimovic

[16] Vikram Sreekanti, Chenggang Wu, Xiayue Charles Lin, Johann Schleier-Smith, Joseph E. Gonzalez, Joseph M. Hellerstein, and Alexey Tumanov. 2020. Cloudburst: Stateful Functions-as-a-Service. *Proceedings of the VLDB Endowment* 13, 12 (2020), 2438–2452. https://doi.org/10.14778/3407790.3407836

[17] Benjamin Carver, Jingyuan Zhang, Ao Wang, Ali Anwar, Panruo Wu, and Yue Cheng. 2020. Wukong: A Scalable and Locality-Enhanced Framework for Serverless Parallel Computing. In *Proceedings of the 11th ACM Symposium on Cloud Computing (SoCC '20)*, 1–15. https://doi.org/10.1145/3419111.3421286

[18] Haoran Zhang, Adney Cardoza, Peter Baile Chen, Sebastian Angel, and Vincent Liu. 2020. Fault-tolerant and Transactional Stateful Serverless Workflows. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*, 1187–1204. https://www.usenix.org/conference/osdi20/presentation/zhang-haoran

[19] Zhipeng Jia and Emmett Witchel. 2021. Boki: Stateful Serverless Computing with Shared Logs. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*, 691–707. https://doi.org/10.1145/3477132.3483541

[20] Tingjia Cao, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Tyler Caraza-Harter. 2025. Making Serverless Pay-For-Use a Reality with Leopard. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '25)*, April 28 - 30, 2025, Philadelphia, PA, USA. USENIX Association, Berkeley, CA, USA.

[21] Xinquan Cai, Qianlong Sang, Chuang Hu, Yili Gong, Kun Suo, Xiaobo Zhou, and Dazhao Cheng. 2024. Incendio: Priority-Based Scheduling for Alleviating Cold Start in Serverless Computing. *IEEE Transactions on Computers* 73, 7 (2024), 1780–1794. https://doi.org/10.1109/TC.2024.3386063

[22] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*, July 10 - 12, 2024, Santa Clara, CA, USA. USENIX Association, Berkeley, CA, USA, 135–153.

[23] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the Web Up to Speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '17)*, June 18 - 23, 2017, Barcelona, Spain. ACM, New York, NY, USA, 185–200. https://doi.org/10.1145/3062341.3062363

[24] Ksenia Bestuzheva, Mathieu Besançon, Wei-Kun Chen, Antonia Chmiela, Tim Donkiewicz, Jasper van Doornmalen, Leon Eifler, Oliver Gaul, Gerald Gamrath, Ambros Gleixner, Leona Gottwald, Christoph Graczyk, Katrin Halbig, Alexander Hoen, Christopher Hojny, Rolf van der Hulst, Thorsten Koch, Marco Lübbecke, Stephen J. Maher, Frederic Matter, Erik Mühmer, Benjamin Müller, Marc E. Pfetsch, Daniel Rehfeldt, Steffan Schlein, Franziska Schlösser, Felipe Serrano, Yuji Shinano, Boro Sofranac, Mark Turner, Stefan Vigerske, Fabian Wegscheider, Philipp Wellner, Dieter Weninger, and Jakob Witzig. 2023. Enabling research through the SCIP optimization suite 8.0. *Mathematical Programming Computation* 15, 4 (2023), 769–821. https://doi.org/10.1007/s12532-023-00232-x

[25] Ashraf Mahgoub, Karthick Shankar, Subrata Mitra, Ana Klimovic, Somali Chaterji, and Saurabh Bagchi. 2021. SONIC: Application-aware Data Passing for Chained Serverless Applications. In *Proceedings of the 2021 USENIX Annual Technical Conference (USENIX ATC '21)*, July 14 - 16, 2021. USENIX Association, Berkeley, CA, USA, 285–301.

[26] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Eshaan Minocha, Sameh Elnikety, Saurabh Bagchi, and Somali Chaterji. 2022. WISEFUSE: Workload Characterization and DAG Transformation for Serverless Workflows. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6, 2 (2022), 1–28. https://doi.org/10.1145/3530892

[27] Yunshan Jia, Chao Jin, Qing Li, Xuanzhe Liu, and Xin Jin. 2025. FaaSPR: Latency-Oriented Placement and Routing Optimization for Serverless Workflow Processing. *IEEE/ACM Transactions on Networking* 33, 4 (2025), 2063–2078. https://doi.org/10.1109/TON.2025.3552407

[28] Gideon Juve, Ann L. Chervenak, Ewa Deelman, Shishir Bharathi, Gaurang Mehta, and Karan Vahi. 2013. Characterizing and Profiling Scientific Workflows. *Future Generation Computer Systems* 29, 3 (2013), 682–692. https://doi.org/10.1016/j.future.2012.08.015

[29] 2021. Collection of Workflow Execution Instances for the Pegasus Workflow Management System. https://github.com/wfcommons/pegasus-instances